

Sistema de arquivos remoto

Maykon Marcos Júnior (22102199)
Gabriel Turatti Andrade (21105479)
Tális Breda (22102202)

Universidade Federal de Santa Catarina (UFSC)
INE5418 - Computação distribuída

6 de outubro de 2025

1 Decisões de implementação

1.1 Linguagens e tecnologias

A aplicação cliente-servidor principal do projeto foi implementada utilizando a linguagem Rust, escolhida por conta da experiência de membros da equipe com ela e por conta da eficiência com a qual ela lida com concorrência de dados. A linguagem escolhida para a segunda aplicação cliente foi o C++, por conta da semelhança com o Rust, pela liberdade de manipulação de dados oferecida, e por ser uma linguagem muito utilizada no dia a dia.

1.2 Estratégias

A forma como a comunicação foi feita se baseou em estruturas de requisição e resposta, os clientes podiam enviar quatro tipos diferentes de requisições, sendo elas as de abrir, ler, escrever e fechar. Cada uma dessas requisições utiliza seus campos para enviar informações relevantes como posição do arquivo.

O servidor possui três tipos de respostas, um Okay, que contém informação relevante para a requisição do cliente, uma mensagem de Erro, para quando não foi possível realizar a requisição, e uma mensagem de Atualização de Cache, que é enviada para todo os clientes que possuem um arquivo em aberto sempre que aquele arquivo é escrito.

Em determinados momentos durante o desenvolvimento, foi necessário fazer algumas escolhas relacionadas a implementação do sistema.

Uma delas foi quanto ao tratamento de requisições de invalidação de cache enviadas do servidor para o cliente. Houve duas estratégias consideradas:

- **Thread dedicada:** Implementação que envolve uma thread separada dedicada a escutar requisições de invalidação de cache. Problemas:

- Essa thread atribui o endereço do cliente ao socket (usando `bind()`), impedindo que a thread principal faça essa atribuição também.
- O servidor utiliza a combinação endereço:porta de um cliente para identificá-lo unicamente
- Como a thread principal não consegue executar o `bind()`, todas as requisições enviadas por ela possuem uma porta diferente, escolhida pelo SO. Logo, o servidor não consegue identificar o cliente unicamente, causando problemas de coerência
- **Tratamento na leitura:** Essa solução não possui uma thread dedicada para as requisições de invalidação de cache. Ao invés disso, sempre que o cliente tenta fazer uma requisição de leitura, ele verifica a fila de requisições de invalidação, e invalida o que for necessário antes de continuar a busca na cache. Problemas:
 - Requisições de leitura podem demorar mais que o necessário se o volume for muito grande, pois será preciso verificar e tratar requisições antes de seguir o fluxo a cada tentativa de leitura
 - Ter uma cache desatualizada não é uma boa prática, mesmo que ela vá ser atualizada sempre que for utilizada.

Apesar do possível problema de desempenho, essa estratégia foi a escolhida por conta do sistema de identificação de clientes presente no servidor, que não pode ser usado caso haja uma thread dedicada, como explicado anteriormente.

1.3 Padrão de projeto

O Design Pattern escolhido foi o Padrão de Limitação ([Throttling](#)).

Ele é importante para evitar sobrecarga, especialmente no contexto de um servidor single-thread sem limitação no número de clientes.

O servidor possui um hashmap onde ele registra o endereço, timestamp do 1º request e número de requisições de cada cliente. Após receber uma requisição de um novo cliente, ele insere o endereço, junto com o instante atual (calculado desde o instante que o servidor iniciou) e o valor 1. Sempre que o mesmo endereço faz novas requisições, o contador de requisições é incrementado até atingir o limite (para exemplificação nos testes, 5). Então, é verificado se o tempo entre o 1º request e o atual é menor que um intervalo máximo (como 1 segundo). Se sim, o requisito é negado e uma mensagem de erro retornada. Se não, a entrada no hashmap é resetada, inclusive o timestamp.

2 Compilação e execução

O projeto é compilado utilizando `cargo`, que está presente na instalação padrão do Rust (via <https://rust-lang.org/tools/install/>).

Para compilar, basta executar o comando `cargo build`.

Para executar, foram criadas duas formas:

- **Testes automatizados:** Para executar os teste automatizados criados para o sistema, basta executar o comando `cargo run <n>`, onde n é o número de agentes.

- **Execução manual:** Caso haja necessidade, é possível executar o programa manualmente, controlando vários clientes ao mesmo tempo. Para isso, basta executar o comando `cargo run` com determinados argumentos. Por exemplo:

- `cargo run server` Inicializa o servidor
- `cargo run 0` Inicializa um cliente com ID 0

Vários clientes podem ser criados em conjunto com o servidor apenas abrindo múltiplos terminais e criando diferentes clientes. Isso também pode ser feito em computadores diferentes de uma mesma rede.

3 Exemplos de saída

Os testes da aplicação foram criados via agentes, que são intermediários entre o `main.rs` (arquivo principal do Rust) e a classe `Client`, que represente o cliente da aplicação. Os agentes possuem os mesmos métodos dos `clients`.

Os resultados de alguns testes estarão em imagens neste relatório, porém os resultados de todos estarão na pasta `/test_results`, dentro da pasta raiz do projeto.

3.1 Escrita e leitura simples

Este primeiro teste é para as funcionalidades básicas da aplicação: escrever e ler arquivos. Ele consiste no seguinte:

1. Criação de dois agentes (0 e 1)
2. Ambos agentes abrem o arquivo `file.txt`
3. Agente 0 escreve uma mensagem no arquivo
4. Agente 1 lê o arquivo
5. **Mensagem lida pelo agente 1 até o tamanho da mensagem do agente 0 precisa ser igual à mensagem do agente 0**

[illegible]

Figura 1: Resultados do teste 1

3.2 Invalidação de cache

Este teste é para o sistema de invalidação de cache. Ele consiste em:

1. Criação de dois agentes (0 e 1)
2. Ambos agentes abrem o arquivo `file.txt`
3. Agente 0 lê o arquivo
4. Agente 1 escreve uma mensagem diferente no arquivo
5. Agente 0 lê o arquivo
6. **A mensagem recebida pela segunda leitura do agente 0 não deve vir da cache; deve ser diferente da primeira leitura; e deve ser igual à mensagem escrita pelo agente 1**

[illegible]

Figura 2: Resultados do teste 2

3.3 Escritas e leituras concorrentes

Este teste é o maior de todos, pois cria 5 agentes e todos escrevem e leem um arquivo simultaneamente. Serve para testar se o servidor lida corretamente com a concorrência de dados e de requisições. Ele consiste, basicamente, em:

1. Criação de cinco agentes (0 a 4)
2. Todos os agentes abrem o arquivo file.txt
3. Todos os agentes escrevem uma mensagem no arquivo, duas vezes seguidas
4. Todos os agentes leem o arquivo
5. Todos os agentes fecham o arquivo

Não será incluída figura para este teste por conta da quantidade de linhas, mas o resultado estará no arquivo `test_results/teste_3.txt`

3.4 Escritas, leituras e coerência de cache

Este teste objetiva testar mais a fundo a coerência de cache, especificamente a questão dos parâmetros de posição e tamanho. O teste consiste em:

1. Criação de dois agentes (0 e 1)
2. Todos os agentes abrem o arquivo file.txt
3. O agente 0 escreve uma mensagem de 512 bytes no arquivo, a partir da posição 0
4. O agente 1 lê 64 bytes do arquivo, a partir da posição 0
5. **A mensagem lida pelo agente 1 deve ser igual aos primeiros 64 bytes da mensagem escrita pelo agente 0**
6. O agente 0 escreve uma nova mensagem de 64 bytes, desta vez a partir da posição 64
7. Repete o passo 4
8. **A mensagem lida pelo agente 1 deve continuar igual aos primeiros 64 bytes da mensagem escrita pelo agente 0; e deve ser obtida da cache, pois apesar de o servidor ter enviado uma invalidação, ela vale apenas para os bytes 64 a 127**
9. O agente 1 lê 64 bytes do arquivo, a partir da posição 64
10. **A mensagem lida pelo agente 1 deve ser igual à mensagem escrita pelo agente 0 no passo 6**

```

Starting Test 4: Large write and read test with cache coherence.
Creating agent 0 on address 127.0.0.1:8081
Creating agent 1 on address 127.0.0.1:8082
[Client 0]: Opening file file.txt with descriptor 0
[SERVER]: Server Received a Open Request with 25 bytes from 127.0.0.1:8081
[Client 0]: Successfully opened file with descriptor 0
[Client 1]: Opening file file.txt with descriptor 0
[SERVER]: Server Received a Open Request with 25 bytes from 127.0.0.1:8082
[Client 1]: Successfully opened file with descriptor 0
[Client 0]: Writing 512 bytes to file descriptor 0 at position 0
[SERVER]: Server Received a Write Request with 529 bytes from 127.0.0.1:8081
[SERVER]: Sending cache invalidation for file 0 from position 0 for 512 bytes
[SERVER]: Sent cache invalidation for file 0 to 127.0.0.1:8081 with 21 bytes
[SERVER]: Sent cache invalidation for file 0 to 127.0.0.1:8082 with 21 bytes
[Client 0]: Successfully wrote to file descriptor 0
[Client 1]: Reading 64 bytes from file descriptor 0 at position 0
[Client 1]: Verifying cache for client at address 127.0.0.1:8082
[Client 1]: Received warning on cache listener, 21 bytes from 127.0.0.1:8080
[Client 1]: Cache invalidation for file descriptor 0 at position 0
[Client 1]: Invalidating cache for file descriptor 0 from position 0 to 512 and size 512
[Client 1]: Cache miss for file descriptor 0 at position 0
[SERVER]: Server Received a Read Request with 17 bytes from 127.0.0.1:8082
[SERVER]: Reading 64 bytes from file 0 at position 0
[Client 1]: Read message from server: "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean"
[Client 0]: Writing 64 bytes to file descriptor 0 at position 64
[SERVER]: Server Received a Write Request with 81 bytes from 127.0.0.1:8081
[SERVER]: Sending cache invalidation for file 0 from position 64 for 64 bytes
[SERVER]: Sent cache invalidation for file 0 to 127.0.0.1:8081 with 21 bytes
[SERVER]: Sent cache invalidation for file 0 to 127.0.0.1:8082 with 21 bytes
[Client 0]: Successfully wrote to file descriptor 0
[Client 1]: Reading 64 bytes from file descriptor 0 at position 0
[Client 1]: Verifying cache for client at address 127.0.0.1:8082
[Client 1]: Received warning on cache listener, 21 bytes from 127.0.0.1:8080
[Client 1]: Cache invalidation for file descriptor 0 at position 64
[Client 1]: Invalidating cache for file descriptor 0 from position 64 to 128 and size 64
[Client 1]: Cache hit for file descriptor 0 at position 0
[Client 1]: Read message from cache: "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean"
[Client 0]: Reading 64 bytes from file descriptor 0 at position 64
[Client 1]: Verifying cache for client at address 127.0.0.1:8082
[Client 1]: Cache miss for file descriptor 0 at position 64
[SERVER]: Server Received a Read Request with 17 bytes from 127.0.0.1:8082
[SERVER]: Reading 64 bytes from file 0 at position 64
[Client 1]: Read message from server: "0000000000000000000000000000000000000000000000000000000000000000"
[Client 0]: Closing file descriptor 0
[SERVER]: Server Received a Close Request with 17 bytes from 127.0.0.1:8081
[Client 0]: Successfully closed file descriptor 0
[Client 1]: Closing file descriptor 0
[SERVER]: Server Received a Close Request with 17 bytes from 127.0.0.1:8082
[Client 1]: Successfully closed file descriptor 0
Test 4 completed successfully.

```

Figura 3: Resultados do teste 4

3.5 Escritas, leituras e coerência de cache pt. 2 - Overlap

Seguindo o molde do teste anterior, este visa verificar se o sistema detecta *overlap* entre a invalidação de cache recebida e a mensagem presente na cache, ou seja, se pelo menos parte dos dados em cache de um arquivo foram invalidados, assim necessitando invalidar o arquivo todo. O teste consiste em:

1. Criação de dois agentes (0 e 1)
2. Todos os agentes abrem o arquivo file.txt
3. O agente 0 escreve uma mensagem de 512 bytes no arquivo, a partir da posição 0
4. O agente 1 lê 64 bytes do arquivo, a partir da posição 0
5. **A mensagem lida pelo agente 1 deve ser igual aos primeiros 64 bytes da mensagem escrita pelo agente 0**
6. O agente 0 escreve uma nova mensagem de 64 bytes, desta vez a partir da posição 32
7. Repete o passo 4
8. **A mensagem lida pelo agente 1 no passo 4 deve ser diferente da mensagem lida no passo 7, pois os últimos 2 bytes foram alterados. Verificando isso, é garantido que a mensagem não foi obtida da cache, significando que o arquivo que estava em cache foi invalidado por conta do overlap**
9. O agente 1 lê 64 bytes do arquivo, a partir da posição 32
10. **A mensagem lida pelo agente 1 deve ser igual à mensagem escrita pelo agente 0 no passo 6, e não deve vir da cache, pois o que está presente na cache é dos bytes 0 a 63, e a leitura é dos bytes 64 a 127**

3.6 Testes com Cliente em C++

Esse teste tem o propósito de mostrar a comunicação entre linguagens de programação diferentes, utilizando um cliente implementado em c++ comunicando com o servidor feito em Rust. No teste é implementado todas as 4 funções principais.

1. Criação de um agente (id 0)
2. Agente faz uma requisição ao server para abrir o arquivo file.txt
3. O agente 0 escreve uma mensagem de 21 bytes no arquivo, "Hello from agent cpp".
4. O agente 0 lê 20 bytes do arquivo, a partir da posição 1
5. A mensagem é colocada na tela, esperasse o retorno "ello from agent cpp", além de mostrar o tamanho da mensagem, 20 bytes.
6. O Agente 0 manda uma requisição para fechar o arquivo.

```
Agent 0 started
Agent 0 trying to open file file.txt
Waiting for response from server...
Response received from server.
Agent 0 opened file file.txt
Agent 0 writing to file 0: Hello from agent cpp

Waiting for response from server...
Response received from server.
Written!
Agent 0 reading from file 0
Waiting for response from server...
Response received from server.
Agent 0 read 20 bytes from file 0:
Read: ello from agent cpp

Waiting for response from server...
Response received from server.
Agent closed file 0
```

Figura 5: Resultados do teste 6 para o cliente.

```
Args len: 2
Arg: ./Distribuida-INE5418
Arg: server
[SERVER]: Server listening on 127.0.0.1:8080
[SERVER]: Server Received a Open Request with 25 bytes from 127.0.0.1:8070
[SERVER]: Server Received a Write Request with 38 bytes from 127.0.0.1:8070
[SERVER]: Sending cache invalidation for file 0 from position 0 for 21 bytes
[SERVER]: Sent cache invalidation for file 0 to 127.0.0.1:8070 with 21 bytes
[SERVER]: Server Received a Read Request with 17 bytes from 127.0.0.1:8070
[SERVER]: Reading 20 bytes from file 0 at position 1
[SERVER]: Server Received a Close Request with 17 bytes from 127.0.0.1:8070
```

Figura 6: Log de atividades do Server para o teste 6.

4 Conclusão

Por conta da simplicidade do projeto, ele possui algumas limitações, que deveriam ser mitigadas caso o sistema fosse mais completo.

Uma delas é quanto à abertura de arquivos pelos clientes. Quando um cliente abre um arquivo, o servidor passa a manter esse cliente em uma lista de observadores. Quando o arquivo é atualizado, cada um dos observadores recebe uma requisição de invalidação de cache, e é removido da lista de observadores. Isso faz com que os clientes precisem constantemente abrir o arquivo caso ele seja atualizado.

Outra limitação é quanto ao uso do protocolo UDP ao invés do TCP para comunicação, pois isso diminui a tolerância a falhas de comunicação entre cliente e servidor.

Ainda, como o servidor usa uma única thread para escutar requisições, haveria perda de desempenho em caso de alto volume de operações, por conta da falta de paralelismo.

A escolha de utilizar a combinação endereço:porta para identificar unicamente um cliente no servidor não é uma boa prática. O correto seria que cada cliente tivesse um ID único gerado aleatoriamente, e enviasse uma requisição ao servidor ao iniciar, similar a um ACK, informando o endereço no qual ele escuta requisições UDP. Essa alteração também permitiria o uso de uma thread específica para invalidações de cache no cliente, já que o endereço completo do cliente já não seria mais relevante para as operações, desde que fosse enviado o ID nas requisições para o servidor.